Question - 1
Python: Transactions Decorators

A list of transactions made by various individuals is provided. Each transaction specifies the person's name and the amount of the transaction, which can be either a credit (positive) or a debit (negative). Implement class decorators to manage these transactions. Create three class decorators:

1. *filter_transactions:* Excludes positive transactions below a threshold of 50.
2. *combine_transactions:* Aggregates transaction amounts for each individual.
3. *sort_transactions:* Orders the combined transactions by the total amount for each individual, maintaining the initial order for equal net amounts.

Note: Exclude individuals with a net balance of zero from the results.

Example

Input:
transactions = [
('Alex', 100),
('Chris', 200),
('Alex', -25),
('Sam', 300),
('Chris', 150),
('Alex', 75),
('Robin', 20)
]

Output:
Chris: $350
Sam: $300
Alex: $150

Filter all transactions less than the threshold of 50, then sum transaction amounts for each person. Report their totals in decreasing order. In this example, Robin's only transaction is lower than the threshold, so it is filtered and the transaction total is 0.

▼ Input Format for Custom Testing

The first line contains an integer, *n*, the size of the array *transactions*.
Each of the next *n* lines contains a tuple *transactions[i]* containing the person's name and transaction amount separated by a space.

▼ Sample Case 0

Sample Input 0

```
5
Chris 50
Sam 75
Robin -100
Chris 25
Alex 150
```

Sample Output 0

```
Alex: $150
Sam: $75
```

```
Chris: $50
Robin: -$100
```

Explanation

Here, Chris's $25 transaction is ignored because it is a positive transaction and less than the threshold value of 50. No person has multiple transactions that need to be added, so the final result shows their balances in descending order.

▼ Sample Case 1

Sample Input 1

```
6
Sam 50
Robin -75
Alex 100
Chris 200
Robin 150
Alex -50
```

Sample Output 1

```
Chris: $200
Robin: $75
Sam: $50
Alex: $50
```

Explanation

No results are filtered here because there are no positive transactions under 50. The rest of the transactions are summed and displayed in descending order of balance. Sam and Alex are in the order they appeared in the original list. (Sam with 50; Robin with 150-75=75; Alex with 100-50=50; and Chris with 200)

Question - 2

Python: Enhanced Date and Time Extractor

Develop a utility that extracts information from custom-formatted date and time strings. The strings can have the following formats, where the month can be represented either numerically (1 to 12) or by its full name (case-insensitive):

- "YYYY-MM-DD HH:mm:ss"
- "YYYY/MM/DD HH:mm:ss"
- "DD Month YYYY HH:mm:ss"

Your function should extract the individual components (*year, month, day, hour, minute,* and *second*) and return them in this format:

"Year: *[year]*, Month: *[month]*, Day: *[day]*, Hour: *[hour]*, Minute: *[minute]*, Second: *[second]*"

- The *month* should be represented numerically (e.g., 1 for January) in the output.
- If the provided string does not match any of the expected formats, raise an *InvalidDateFormat* exception with the message "Invalid date and time format".
- If either the *date* or *time* value is invalid, raise an *InvalidDateFormat* exception with the message "Invalid date or time format".
- If the *month* name is incorrect, raise an *InvalidMonthName* exception with the message "Invalid month name".

Function Description

Complete the function *extract_datetime* with the following parameter(s):

string dateTimeString: the date-time string

Returns

string: the extracted components in the specified format or an error message

Constraints

- The *year* should be a four-digit integer.
- The *month* can be represented either numerically (1 to 12) or by its full name (case-insensitive).
- The *day* should be a two-digit integer.
- The *hour* should be a two-digit integer in 24-hour format (00 to 23).

- The *minute* should be a two-digit integer (00 to 59).
- The *second* should be a two-digit integer (00 to 59).

▼ Input Format For Custom Testing

The first line contains the string *dateTimeString*.

▼ Sample Case 0

Sample Input For Custom Testing

```
2024/02/21 12:30:45
```

Sample Output

```
Year: 2024, Month: 2, Day: 21, Hour: 12, Minute: 30, Second: 45
```

Explanation

The *year, month, day, hour, minute,* and *seconds* are extracted from the above date-time.

▼ Sample Case 1

Sample Input For Custom Testing

```
2024/02/41 25:70:90
```

Sample Output

```
Invalid date or time values
```

Explanation

Because the input is invalid (both the date and time have out-of-range values), the error message "Invalid date or time values" in displayed.

Question - 3

Python: CSS Hex Color Code Matcher

Implement a regular expression in Python that removes comments from CSS code and identifies valid hexadecimal color codes.
In CSS, comments begin with /* and end with */. Valid hexadecimal color codes start with a hash symbol (#) followed by either 3 or 6 hexadecimal digits (0-9, a-f, A-F).

Implement the *css_hex_color_code_matcher(input_text)* function that returns:

1. The input text with all CSS comments removed
2. A list of all valid hexadecimal color codes found in the text

The function should only match color codes that strictly follow the format of a hash symbol followed by exactly 3 or 6 hexadecimal digits.

Constraints

- The length of the text should be less than 25000.
- 0 matches or at least 1 match is guaranteed.

▼ Input Format For Custom Testing

The first line is the text given for CSS color code matching.

▼ Sample Case 0

Sample Input For Custom Testing

```
sample text with color codes like #abc, #123456. The background color is set to #fff. /* This is a comment */
```

Sample Output

```
Modified Text: sample text with color codes like #abc, #123456. The background color is set to #fff.
Matched Hex Color Codes: #123456 #abc #fff
```

Explanation

Here the text is given as above.

▼ Sample Case 1

Sample Input For Custom Testing

```
CSS code with color codes such as #f00, #00ff00, and #fff0fff color. /* Comment */
```

Sample Output

```
Modified Text: CSS code with color codes such as #f00, #00ff00, and blue color.
Matched Hex Color Codes: #00ff00 #f00
```

Explanation

Note that #fff0fff has seven digits, so it is malformed. Malformed color codes are not included in the list of color codes.

Question - 4

Python: Frequency Calculator

Implement a Python script that calculates the frequencies of letters and words in a given string *text*. The results should be sorted first by frequency (descending) and then alphabetically (ascending). Return the top *num* most common letters and words.

Notes:

- Lowercase and uppercase letters/words are considered the same.
- Punctuation should be removed.
- Contractions like "don't" count as two words: "don" and "t".

Example

text = "I don't have to have breakfast this morning."

num = 5

The frequencies of words are ('breakfast', 1), ('don', 1), ('have', 2), ('i', 1), ('morning', 1), ('t', 1), ('this', 1), ('to', 1). Only "have" has a higher frequency than 1, so it is the first in the word frequency list. Others are sorted alphabetically. The expected result is:

```
Top 5 letters:
a: 4
t: 4
e: 3
h: 3
i: 3

Top 5 words:
have: 2
breakfast: 1
don: 1
i: 1
morning: 1
```

Function Description

Complete the function *get_top_letters_and_words* in the editor with the following parameter(s):

string text: the string to analyze

int num: how many most common letters and words should be returned

Return

Return two strings that will print as shown: the *n* most common letters and then the *n* most common words in the string, sorted as mentioned.

The format should be as follows:

Top [num] letters:
[letter]: [frequency]
[letter]: [frequency]
...

Top [num] words:
[word]: [frequency]
[word]: [frequency]
...

Constraints

- The length of *text* is less than 20000.

▼ Input Format For Custom Testing

The first line contains the string *text*.
The second line contains an integer, *num*.

▼ Sample Case 0

Sample Input For Custom Testing

```
Most of the lists and dicts and stuff were fine for me…but when it got to Classes section (Python Code
Challenges: Classes), WOW it got hard. The hints didn't help - looked at the answers and re-read the prompt…
wondering who in the world wrote it because it answered the obvious while skipped the parts that was NOT taught
in the Class section before. Also I found actual spelling typos (like so was spelled "se" etc.)
5
```

Sample Output

```
Top 5 letters:
e: 41
t: 37
s: 32
o: 26
a: 21

Top 5 words:
the: 8
it: 4
and: 3
classes: 2
got: 2
```

Explanation

Here, the number of top common words and letters is 5. Letters and words, with their corresponding frequencies, are displayed.

▼ Sample Case 1

Sample Input For Custom Testing

```
Most of the lists and dicts and stuff were fine for me…but when it got to Classes section
5
```

Sample Output

```
Top 5 letters:
t: 10
s: 9
e: 8
o: 6
```

```
f: 5

Top 5 words:
and: 2
but: 1
classes: 1
dicts: 1
fine: 1
```

Explanation

Here, the number of top common words and letters is 5. Letters and words, with their corresponding frequencies, are displayed.

Question - 5

Python: User Permission Management

Implement a Python user permissions system using closures and decorators.

Requirements

1. Create a function that uses closures to manage user permissions with these roles:
   - Admin: can read, write, and delete
   - Editor: can read and write
   - Viewer: can only read
2. Implement a decorator that:
   - Verifies if a user has the necessary permissions before executing a function
   - Prevents unauthorized actions based on the user's role
   - Returns appropriate error messages for unauthorized access attempts

The solution should demonstrate proper use of both closures to maintain permission state and decorators to enforce permission checks before actions are performed.

Function Description

- *permission_closure()*:
  - Defines a closure function to manage permissions for different roles.
  - *Returns*: A function *check_permission(user_role, action)* that verifies if a given action is allowed for a specified user role based on predefined permissions.
- *authorize(func)*:
  - A decorator function that authorizes actions for a user role.
  - *Parameters*: *func* – the function to be wrapped and checked for authorization.
  - *Returns*: A wrapper that either executes the action or raises a *PermissionError* if authorization fails.
    - Return messages by function:
      - For *read*: returns "Data is: {data}".
      - For *write*: returns "Data updated".
      - For *delete*: returns "Data deleted".
      - If authorization is denied: returns "Permission denied for {user_role} to perform {action} action."

Constraints

- $1 \leq n \leq 50$

▼ Input Format For Custom Testing

The first line is the number of user permission tests.

The user name and role, and the action type with detail, are repeated afterward.

▼ Sample Case 0

Sample Input For Custom Testing

```
2
person1:admin
write:hello world
person2:viewer
read
```

Sample Output

```
Data updated
Data is: hello world
```

Explanation

Here, the number of user permission tests is 2. The user person1's role is admin, and the action is to write. The next user, person2, has the role of viewer, and the action is to read.

▼ Sample Case 1

Sample Input For Custom Testing

```
1
person1:viewer
delete:5
```

Sample Output

```
Permission denied for viewer to perform delete action.
```

Explanation

Here the user person1's role is viewer. The action is to delete the last 5 characters of data. Permission is denied because only admins can delete text.

Question - 6

Python: Library Catalog System

Create a system to manage different types of items in a library catalog, such as books and DVDs. Implement classes for each item type with specific attributes and methods to display information.

The system should utilize magic methods like *__str__* and *__repr__* for enhanced functionality.

Implement the following classes:

1. *Item* Class with attributes for *title*, *author*, and *year*. It should include methods to:
   - Return a formatted string representation using *__str__*: "[title] by [author] ([year])"
   - Return a formal representation using *__repr__*: "Item: [Title] by [Author] ([Year])"
   - Method: *display_info* to display detailed information about the item.

     ```
     Title: [title]
     Author: [author]
     Year: [year]
     ```

2. *Book* Class (inherits from *Item*) with additional attributes for *genre* and *ISBN*. It should override the *display_info* method to include book-specific information.

   ```
   Title: [title]
   Author: [author]
   Year: [year]
   Genre: [genre]
   ISBN: [ISBN]
   ```

3. *DVD* Class (inherits from *Item*) with an additional attribute for *duration*. It should override the *display_info* method to include DVD-specific information.

   ```
   Title: [title]
   Author: [author]
   Year: [year]
   ```

```
    Duration: [duration] minutes
```

Constraints

- The number of items is less than or equal to 50.

▼ Input Format For Custom Testing

The first line indicates the number of items.

The next lines contain a comma-delimited list with the type of item and its attributes, one item per row.

▼ Sample Case 0

Sample Input For Custom Testing

```
2
Book,Python Programming,John Doe,2022,Programming,978-3-16-148410-0
DVD,Movie,Jane Smith,2021,120
```

Sample Output

```
--- Item Information ---
Python Programming by John Doe (2022)
Title: Python Programming
Author: John Doe
Year: 2022
Genre: Programming
ISBN: 978-3-16-148410-0

Details:
Title: Python Programming
Author: John Doe
Year: 2022
Genre: Programming
ISBN: 978-3-16-148410-0

--- Item Information ---
Movie by Jane Smith (2021)
Title: Movie
Author: Jane Smith
Year: 2021
Duration: 120 minutes

Details:
Title: Movie
Year: 2021
Duration: 120
```

Explanation

There are two items.

The first type of item is a book, with attributes of the *Book* class.

The second type of item is a DVD, with attributes of the *DVD* class.

▼ Sample Case 1

Sample Input For Custom Testing

```
1
DVD,Movie,John Doe,2019,120
```

Sample Output

```
--- Item Information ---
Movie by John Doe (2019)
Title: Movie
Author: John Doe
Year: 2019
Duration: 120 minutes

Details:
```

```
Title: Movie
Year: 2019
Duration: 120
```

Explanation

There is one item.

It is a DVD with attributes of a *DVD* instance.

Question - 7

Python: Bank Vault Access Manager

Implement a program that manages access to a bank vault, allowing authorized employees to perform specific activities inside the vault. The system should implement two classes: *VaultAccessControl* and *BankVaultAccessManager*.

VaultAccessControl manages the list of authorized employees and controls access to the vault.

Methods:

- *__init__(self, authorized_employees, all_employees)*: Initializes with a list of authorized employees and all employees
- *grant_access(self, employee_id)*: Grants access if the employee ID is authorized
- *release_access(self)*: Releases the access lock

BankVaultAccessManager provides a context manager interface for vault access.

Methods:

- *__init__(self, employee_id, access_control)*: Initializes with an employee ID and access control instance
- *__enter__(self)*: Grants access when entering the context manager
- *__exit__(self, exc_type, exc_value, traceback)*: Releases access; raises an exception if access was not granted
- *perform_activity(self, activity)*: Executes a specified activity within the vault if access is granted

The system should handle different access scenarios with appropriate error messages:

- Unauthorized access attempts with "Unauthorized access. Employee ID {employee_id} not authorized to access the vault."
- Unrecognized employee IDs with "Unauthorized access. Employee ID {employee_id} not recognized."
- Successful activities with "Authorized Access for {self.employee_id}. Performed activity inside the bank vault: {activity}"

Constraints

- $1 \le n \le 100$, where n is the total number of employees
- $1 \le m \le 10$, where m is the total number of authorized employees
- $1 \le q \le 10$, where q is the total number of access requests to the vault

▼ Input Format For Custom Testing

The first line contains an integer, n, the number of employees.

The second line is all the employee IDs, separated by commas.

The next line contains an integer, m, the number of authorized employees.

The next line is authorized employee IDs, separated by commas.

The next line contains an integer, q, the number of users trying to access the vault.

The next line is users trying to access the vault, separated by commas.

▼ Sample Case 0

Sample Input For Custom Testing

```
5
EMP1234,EMP5678,EMP9101,EMP2344,EMP46532
1
EMP9101
4
EMP1234,EMP5678,EMP9101,EMP6532
```

Sample Output

```
Error occurred: Unauthorized access. Employee ID EMP1234 not authorized to access the vault.
Error occurred: Unauthorized access. Employee ID EMP5678 not authorized to access the vault.
Authorized Access for EMP9101. Performed activity inside the bank vault: Counting cash
Error occurred: Unauthorized access. Employee ID EMP6532 not recognized.
```

Explanation

Five employee IDs are given, and EMP9101 is authorized. Three existing employees attempt access, but two are not authorized. Finally, a non-existing ID is tested.

▼ Sample Case 1

Sample Input For Custom Testing

```
6
EMP1234,EMP5678,EMP9101,EMP321,EMP5678,EMP2021
3
EMP1234,EMP5678,EMP9101
2
EMP1234,EMP5796
```

Sample Output

```
Authorized Access for EMP1234. Performed activity inside the bank vault: Counting cash
Error occurred: Unauthorized access. Employee ID EMP5796 not recognized.
```

Explanation

Lists of employees and authorized employees are created. Two access attempts are made, and appropriate responses are returned.

Question - 8

Python: Banking Transaction Exception

Create an advanced Python program to simulate banking transactions with various features, including transaction

Create a Python program to simulate banking transactions with transaction history, security measures, and error handling features.

Implement two classes:

1. *Transaction* Class
   - *__init__(self, transaction_type, amount, balance)*: Initializes a new transaction
   - *__str(self)__*: Returns a string representation of the transaction in the form "Type: {self.transaction_type}, Amount: {self.amount}, Balance: {self.balance}"
2. *BankAccount* Class
   - *__init__(self, initial_balance)*: Initializes an account with specified balance
   - *process_transaction(self, transaction_amount, transaction_type)*: Processes a transaction

Initial Conditions

- Account balance starts at 1500.0

Transaction Types

- deposit:
  - Adds the transaction amount to the balance
  - Updates transaction history
  - Returns:
    - "Transaction successful!"
    - "Updated account balance: {self.balance}"
- withdraw:
  - Deducts the transaction amount if sufficient funds are available
  - Updates transaction history
  - If funds are insufficient, raises an *InsufficientFundsError*
  - Returns:
    - "Transaction successful!"
    - "Updated account balance: {self.balance}"
- view_transaction_history:

- Displays transaction history, with each transaction shown in the format "Type: {self.transaction_type}, Amount: {self.amount}, Balance: {self.balance}"

Custom Exceptions

- *InsufficientFundsError*:
  - Raised when withdrawal exceeds available funds
  - Message: "Not enough funds to withdraw {transaction_amount}. Current balance: {balance}"
  - status_code: 400
- *InvalidTransactionType*:
  - Raised for invalid transaction types
  - Message: "Invalid transaction type '{transaction_type}'. Accepted types: deposit, withdraw, view_transaction_history"
  - status_code: 400

Constraints

- All transaction amounts are positive floats in the form of a string.
- The account balance is a non-negative float.
- $1 \leq n \leq 1000$

▼ Input Format For Custom Testing

The first line contains an integer, *n*, representing the number of transactions.

For the next *n* lines:

If the transaction type is not view_transaction_history, the line contains the transaction type followed by the transaction amount in the next line.

If the transaction type is view_transaction_history, the line contains only the transaction type.

▼ Sample Case 0

Sample Input For Custom Testing

```
3
deposit
130000.0
withdraw
3400.0
view_transaction_history
```

Sample Output

```
Transaction successful!
Updated account balance: 131500.0
Transaction successful!
Updated account balance: 128100.0
Type: deposit, Amount: 130000.0, Balance: 131500.0
Type: withdraw, Amount: 3400.0, Balance: 128100.0
```

Explanation

First, 130000.0 is deposited, bringing the balance to 131500.0, and then 3400.0 is withdrawn. Finally, the transaction history is viewed.

▼ Sample Case 1

Sample Input For Custom Testing

```
3
deposit
500.0
withdraw
1000.0
invalid_type
```

Sample Output

```
Transaction successful!
Updated account balance: 2000.0
Transaction successful!
```

```
Updated account balance: 1000.0
Error (400): Invalid transaction type 'invalid_type'. Accepted types: deposit, withdraw,
view_transaction_history.
```

Explanation

First, 500.0 is deposited so the balance is 2000. Then 1,000.0 is withdrawn. Finally, "invalid_type" is added as a transaction type, so the error is shown.

Question - 9

Python: Secure Login System

Implement a Python program using closures to create a secure login system. The closure should:

1. Store provided user credentials
2. Allow access only when the correct username and password are entered
3. Log each successful and failed login attempt

Function Description

Complete the function *login_system* in the editor with the following parameters:

dict login_attempts: a dictionary with *'success'* and *'failed'* as keys and their counts as the values

Returns

- *function secure_login(string username, string password)*:
  - Manages the login process by checking the provided username and password against stored credentials.
  - Logs the attempt as either successful or failed.
  - Return Values:
    - If credentials are correct: *"Welcome, {username}!"*
    - If credentials are incorrect: *"Failed login attempt for {username}"*
- *function get_login_attempts()*:
  - This returns the *login_attempts* dictionary, which has keys *'success'* and *'failed'* to track the number of successful and failed login attempts.

Constraints

- The number of users is less than or equal to 30.

▼ Input Format For Custom Testing

The integer in the first line denotes the number of users.

The following lines contain the *{username}, {password}* for each of the users.

The integer in the following line denotes the number of users trying to log in.

The following lines contain *{username}, {password}* for login attempts.

▼ Sample Case 0

Sample Input For Custom Testing

```
4
admin,3rTb9ZaYqF6vP2dQ8wKmXsNcRJ5tL7hP2MqU
user1,1aXc8YbOpR4eT9sD6uJnWqAcLJ3vK8eF1LpM
user2,5sFb4XnVqR2pT6dE9wKmSsNcZJ8tL5hP9MqU
user3,9aBc8YdOpR4eT9sD6uJnWqAcLJ3vK8eF1LpM
2
admin,3rTb9ZaYqF6vP2dQ8wKmXsNcRJ5tL7hP2MqU
user1,1aXc8YbOpR4eT9sD6uJnWqAcLJ3vK8eF1LpM
```

Sample Output

```
Welcome, admin!
Welcome, user1!
Login Attempts - Successful: 2 Failed: 0
```

Explanation

There are usernames and passwords for 4 users, and two login attempts. The credentials are correct for both users, so their welcome messages are displayed. Finally, the login attempt count is displayed.

▼ Sample Case 1

Sample Input For Custom Testing

```
4
admin,3rTb9ZaYqF6vP2dQ8wKmXsNcRJ5tL7hP2MqU
user1,1aXc8YbOpR4eT9sD6uJnWqAcLJ3vK8eF1LpM
user2,5sFb4XnVqR2pT6dE9wKmSsNcZJ8tL5hP9MqU
user3,9aBc8YdOpR4eT9sD6uJnWqAcLJ3vK8eF1LpM
1
user,pass123
```

Sample Output

```
Failed login attempt for user
Login Attempts - Successful: 0 Failed: 1
```

Explanation

There are credentials for 4 users and one failed login attempt.

Question - 10

Python: Weather Forecasting Service

Implement a weather forecasting service in Python with custom exceptions for error handling:

- TemperatureOutOfRange
- InvalidTemperatureData
- InvalidForecastData
- InvalidCityName

The implementation should demonstrate proper exception handling techniques and provide detailed feedback to users when errors occur during weather data processing.

Class Functionality:

- *WeatherService* processes weather data for cities, handling errors related to temperature and forecast data by raising specific exceptions as needed. It supports querying weather data for a city based on specified parameters.

1. *weather_data_enum(data):*
   - Parameters:
     - *data* (dictionary): Data used to create an enumeration.
   - Return Value: Enumeration object created from the provided data.
2. *process_weather_data(WeatherData, city, query):*
   - Parameters:
     - *WeatherData* (enum): Enum containing weather data for various cities.
     - *city* (string): The city whose weather data is queried.
     - *query* (string): Specific parameter queried (e.g., temperature or forecast).
   - Return Value: Returns a string with the queried weather data: *"The {query} for {city} is {weather_value}"*.
   - Exceptions Raised with Messages:
     - *InvalidTemperatureData*: Raised if temperature data is missing or invalid. Message: *"Invalid or missing {query} data"*.
     - *InvalidForecastData*: Raised if forecast data is missing or invalid. Message: *"Invalid or missing {query} data"*.
     - *InvalidCityName*: Raised if the city name is not in the data. Message: *"Invalid city name {city}"*.
     - *TemperatureOutOfRange*: Raised if temperature is outside the valid range (-20 to 50 inclusive). Message: *"Temperature {weather_value} is out of range"*.

Constraints

- There are 50 or fewer cities with their weather data.

▼ Input Format For Custom Testing

The first line is the number of weather data points that are used for service.
The weather data is given afterward, following the format *{city}, {temperature}, {forecast}*.
The next line is the number of queries.
Query information follows in the format *{city}, {weather_type(temperature or forecast)}*.

▼ Sample Case 0

Sample Input For Custom Testing

```
7
New York,25,
London,18,
Paris,,sunny
Madrid,29,
Berlin,,snowfall
Ulaanbaatar,-30,
Sydney,30,sunny
2
New York,temperature
Paris,temperature
```

Sample Output

```
The temperature for New York is 25
Error: Invalid or missing temperature data
```

Explanation
There are 7 weather data points given, then 2 queries. The result for the first query displays the temperature in New York, but Paris is missing temperature data.

▼ Sample Case 1

Sample Input For Custom Testing

```
7
New York,55,
London,,sunny
Paris,27,
Madrid,,humid
Berlin,14,
Ulaanbaatar,-30,
Sydney,30,sunny
3
New York,temperature
Paris,forecast
xyz,temperature
```

Sample Output

```
Error: Temperature 55 is out of range
Error: Invalid or missing forecast data
Error: Invalid city name xyz
```

Explanation
Each query raises the appropriate error.

Question - 11
Python: Book Popularity Tracker

Implement a Python application for managing a bookstore's inventory. The application should allow adding books, finding highest-rated books, and recommending books by genre.

There are two classes:

1. *Book* class with attributes:
   - *title (string)*: the book's title
   - *genre (string)*: the book's genre
   - *rating (float)*: the customer rating
2. *LibraryManagement* class with methods:
   - *add_book(Book book)*: adds a new book to the inventory
   - *get_highest_rated_books(int x)*: returns the *x* highest-rated books, sorted by *rating* (descending) and *title* (descending) for ties
   - *recommend_top_book_by_genre(string genre)*: returns the highest-rated book in the specified genre, or None if no books exist in that genre

The code for reading input, calling methods, and producing output is already provided.

Constraints

- The number of recommended books is less than or equal to the number of books in the book list.

▼ Input Format For Custom Testing

The first line contains an integer, *n,* the number of books.

Each of the next *n* lines contains a comma-delimited list of *title, genre,* and *rating*.

The next line contains an integer, the number of books to recommend.

The last line contains a string, the genre to recommend from.

▼ Sample Case 0

Sample Input For Custom Testing

```
3
Python Programming,Programming,4.5
Data Science Essentials,Data Science,3.1
C++ Programming,Programming,4.3
1
Programming
```

Sample Output

```
Highest Rated Book: Python Programming, Rating: 4.5
Top Recommended Book in Programming Genre: Python Programming, Rating: 4.5
```

Explanation

Here the number of recommended books is 1.

The specified genre is "Programming".

▼ Sample Case 1

Sample Input For Custom Testing

```
5
Python Programming,Programming,4.5
Data Science Essentials,Data Science,3.1
C++ Programming,program,4.3
Algorithm,Algorithm,2.9
Java Programming,Programming,4.7
3
Programming
```

Sample Output

```
Highest Rated Book: Java Programming, Rating: 4.7
Highest Rated Book: Python Programming, Rating: 4.5
Highest Rated Book: C++ Programming, Rating: 4.3
Top Recommended Book in Programming Genre: Java Programming, Rating: 4.7
```

Explanation

Here the number of recommended books is 3.

The specified genre is "Programming".

Question - 12
Python: Event Reservation

Implement a *ShowBooking* service for a theater with multiple auditoriums. Each auditorium has a fixed seating capacity and can host a single event per day. The service handles the following:

- Stores which event will take place in an auditorium of the theater
- Avoids conflicts when multiple customers are booking tickets for an event

Implement two classes:

1. Complete the *Repo* class that implements the *IRepo* interface, which stores booking information:

- *registerEventInAuditorium(auditoriumNumber, eventId, capacity)*: Create and store an *Auditorium* object from the parameters.
- *getAuditoriumDetailsForEvent(eventId)*: Return the *Auditorium* object for the event.
- *getNumberOfSeatsBookedOrUnderBookingForEvent(eventId)*: Return the sum of tickets booked and tickets under booking for the event.
- *addUnderBooking(eventId, userId)*: Store information that the user is booking an event.
- *removeUnderBooking(eventId, userId)*: Remove information that the user is booking an event.
- *addSuccessfulBooking(eventId, userId)*: Store information that the user has successfully booked an event.

2. Complete the *BookingService* class that implements the *IBookingService* interface:

- *startBookingProcess(eventId, userId)*: If the sum of booked and under-booking tickets is less than the auditorium capacity, start booking for the user and return true; otherwise, return false.
- *confirmBookingStatus(eventId, userId, bookingSuccessful)*: If the booking is successful, update its status and return a *BookingConclusion* object with the auditorium number and confirmation status.

The driver code creates an object of the class *Repo* and passes it to the constructor of the class *BookingService.* The driver code uses this object to call *registerEventInAuditorium(int auditoriumNumber, int eventId, int capacity)* and register information regarding events and auditoriums. It also handles reading input from the console and calling appropriate functions.

Constraints

- $1 \le totalNumberOfRequests \le 10^5$
- $1 \le userId \le 10^5$
- $1 \le eventId \le 10^5$
- $1 \le auditoriumNumber \le 10^5$

▼ Input Format For Custom Testing

The first line contains an integer, *n*, the number of auditoriums in the theater.

The second line contains *n* space-separated integers describing the array *eventIdInAuditorum,* where each element represents the event with *eventId* that will take place in the ith auditorium.
The third line contains *n* space-separated integers describing the array *capacity[],* where each element represents the capacity of the ith auditorium.
The fourth line contains an integer *totalNumberOfRequests.*
Each line *i* of the next *totalNumberOfRequests* lines describes each request made to the *BookingService class* to be called, along with its parameters, in order.

▼ Sample Case 0

Sample Input For Custom Testing

```
STDIN                    Function
-----                    --------
6                        n = 6
4 5 7 1 2 3              eventIdInAuditorium = [4, 5, 7, 1, 2, 3]
2 2 2 2 2 2              capacity = [2, 2, 2, 2, 2, 2]
5                        totalNumberOfRequests = 5
startBookingProcess 7 1  first request
startBookingProcess 7 2
startBookingProcess 7 3
```

```
confirmBookingStatus 7 1 true
startBookingProcess 7 4
```

Sample Output

```
true
true
false
successful 2
false
```

Explanation

There are 6 auditoriums in the theater, each with a capacity of 2 seats.

- *startBookingProcess 7 1 :* A user with *userId* 1 is trying to book a ticket for the event with *eventId* 7.
- *startBookingProcess 7 2 :* A user with *userId 2* is trying to book a ticket for the event with *eventId* 7.
- *startBookingProcess 7 3 :* A user with *userId 3* is trying to book a ticket for the event with *eventId* 7. There are already users with *userIds (1, 2),* and the capacity of the auditorium that hosts this event (*auditoriumNumber 3*) is 2. So, it returns *false.*
- *confirmBookingStatus 7 1 true* : Booking of *userId 1* for *eventId* 7 is successful. The event takes place in *auditoriumNumber 2,* so the code stub prints "*successful 2"*
- *startBookingProcess 7 4:* Booking *userId* 4 for *eventId 7* cannot be started because there is one user (*userId 1*) who has already booked a seat, and another user (*userId 2*) is trying to book a seat, but the *capacity* of the auditorium is *2*. So, the code stub prints "*false"*.

Question - 13

Python: Menu Recommendation

Create a system that suggests menu items for a restaurant based on the following rules:

1. If the head chef designates a dish as the "deal of the day," recommend that dish.
2. If there is no "deal of the day," recommend the dish with the highest average rating.
3. If there is a "deal of the day" but it is out of stock, recommend the highest-rated in-stock item.
4. If two items share the same highest average rating, recommend the one with the lower item ID.

Notes:

- An out-of-stock item loses its "deal of the day" status permanently, even if restocked later.
- Items with no ratings are considered to have a rating of zero.
- The average rating is calculated by dividing the sum of all ratings for an item by the number of ratings it has received.

Implement the class *MenuRecommendation* that inherits from the abstract base class *IMenuRecommendation* with the following methods:

- *addItem(int itemId, String displayName)*: Create and store a *MenuItem* object from the given information. The definition of class *MenuItem* is given in the code stub.
- *getRecommendedItem()*: Return the recommended *MenuItem*. If there is no such *MenuItem*, return *null*.
- *outOfStockItem(int itemId)*: Mark *itemId* as out of stock.
- *restockItem(int itemId):* Mark *itemId* as back in stock.
- *makeDealOfTheDayItem(int itemId):* Mark *itemId* as the deal of the day and the recommended item.
- *rateItem(int itemId, int rating):* A user rated the item with *itemId* as *rating* number of points.

The driver code takes care of input and calls the relevant functions. There are *totalNumberOfRequests,* and each of the next lines is a request that is one of 6 types of function call.

Constraints

- $1 \le totalNumberOfRequests \le 10^3$
- $1 \le itemId \le 10^5$
- $1 \le rating \le 5$
- $1 \le |displayName| \le 10$

▼ **Input Format For Custom Testing**

The first line contains an integer, *totalNumberOfRequests,* the number of requests.

Each *i* of the next *totalNumberOfRequests* contains a request described above.

▼ **Sample Case 0**

Sample Input For Custom Testing

```
8
getRecommendedItem
addItem 1 Item1
rateItem 1 5
getRecommendedItem
outOfStockItem 1
rateItem 1 4
rateItem 1 4
getRecommendedItem
```

Sample Output

```
N/A
1 Item1 Rating: 5.0
N/A
```

Explanation

- *getRecommendedItem* - there are no item entries, so this outputs 'N/A'
- *addItem 1 Item1* - Adds *Item1* with *itemId* 1.
- rateItem 1 5 - Adds a rating of 5 to *Item1*.
- *getRecommendedItem* - there is only 1 item added yet with 1 rating of 5.
- *outOfStockItem 1* - marks *Item1* as out of stock
- *rateItem 1 4* - Adds a rating of 4 to *Item1*.
- *rateItem 1 4* - Adds a rating of 4 to *Item1*.
- getRecommendedItem - There are no items in stock to recommend.

Question - 14

DB Context

Implement a class called *database_context_manager* that acts as a context manager for database connections.

The context manager should:

- Create a database connection when entering the context by calling the *_init_db()* method (already defined in the *Database* class)
- Close the connection when exiting the context by calling the *_close_db()* method (also defined in the *Database* class)

Given a string *username* consisting of lowercase English letters, define and complete the *database_context_manager class*.

For a successful simulation:

- The input string username must be passed to *_init_db()* as an argument.
- The connection string returned from *_init_db()* must be passed to *_close_db()* as an argument to close the connection.

Example

Given input *username* = "Sam", if the connection string returned by *_init_db()* is "abc":

- "Sam" must be passed as an argument to *_init_db()*
- "abc" must be passed to *_close_db()*

Function Description

Complete the class *database_context_manager* in the editor with the following parameter(s):

string username: the string to pass to *_init_db()*

Constraints

- $1 \le |\,username\,| \le 10^5$
- *username* consists of lowercase English letters.

▼ **Input Format For Custom Testing**

The first line and the only line of the test case contains a string, *username*.

▼ **Sample Case 0**

Sample Input For Custom Testing

```
Robin
```

Sample Output

```
Username matched, database connection initialized
Database running...
Connection string matched, database connection closed
```

Explanation

Given, *username = "Robin",* the steps processed are:

- Implement the class *database_context_manager.*
- Call the *_init_db()* method with the correct parameter. Output received is "Username matched, database connection initialized".
- The output received is "Database running..." to signal that the database is currently in a running state.
- Call the *_close_db()* method with the correct parameter. The output received is "Connection string matched, database connection closed".

▼ **Sample Case 1**

Sample Input For Custom Testing

```
Sam
```

Sample Output

```
Username matched, database connection initialized
Database running...
Connection string matched, database connection closed
```

Explanation

Given, *username = "Sam"*, the steps processed are:

- Implement the class *database_context_manager.*
- Call the *_init_db()* method with the correct parameter. Output received is "Username matched, database connection initialized".
- The output received is "Database running..." to signal that the database is currently in a running state.
- Call the *_close_db()* method with the correct parameter. The output received is "Connection string matched, database connection closed".

Question - 15

Mixins

Implement two mixin classes that add functionality to Python classes:

1. *DictMixin:* Adds a *to_dict* method that converts a Python class to a dictionary
2. *JSONMixin:* Adds a *to_json* method that converts a Python class to JSON

These mixins should:

- Only convert object attributes that do not start with '_' (underscore)
- Raise a *TypeError* with the message "Object is not JSON serializable" if a class cannot be converted to JSON

Example

```
class MyClass(DictMixin, JSONMixin):
    def __init__(self, name, data, secret):
        self.name = name
        self.data = data
        self._secret = secret

obj = MyClass("abc", 10, "secret")
```

- *obj.to_dict()* should return {'name': 'abc', 'data': 10}
- *obj.to_json()* should return {"name": "abc", "data": 10}

Implementation Description

Complete the class *DictMixin* and *JSONMixin* in the editor below.

Constraints

- 1 ≤ |*name*|, |*secret*| ≤ 20
- *data* is either a set or an integer with values in the range from 1 to 100 (inclusive).
- *name* and *secret* contain lowercase English letters only.

▼ Input Format For Custom Testing

The first line contains a string, *name*.
The next line contains a string, *dataType*, denoting the data type of *data*.
The following lines contain either an integer if *dataType* is *int*, or a length of a list and then the list of numbers if *dataType* is *set*.

The last line contains a string, *secret*.

▼ Sample Case 0

Sample Input For Custom Testing

```
STDIN       Function
-----       --------
alex        name = 'alex'
set         dataType = 'set'
2           data size = 2
12          data = {12, 13}
13
secretkey   secret = 'secretkey'
```

Sample Output

```
{'name': 'alex', 'data': {12, 13}}
Object is not JSON serializable
```

Explanation

In this example, *obj.to_dict()* should return {'name': 'alex', 'data': {12, 13}}, and *obj.to_json()* should return *TypeError* with message "Object is not JSON serializable".

▼ Sample Case 1

Sample Input For Custom Testing

```
alex
int
10
key
```

Sample Output

```
{'name': 'alex', 'data': 10}
{"name": "alex", "data": 10}
```

Explanation

Here, *obj.to_dict()* should return {'name': 'alex', 'data': 10}, and *obj.to_json()* should return {"name": "alex", "data": 10}.

Question - 16
Decorator Timing

Implement a Python decorator called *timeit* that measures the execution time of a function. The decorator takes a single argument, *func*, which is the function to be timed.

When the decorated function is called, the decorator should:

1. Measure the execution time in milliseconds
2. Push the execution time to a list called *execution_time* (already declared in the global scope)
3. Return the same value that the original function would return

Example

function_name = "delay_max"
args = [3, 4, 1]
time_delay = 150

Decorate the function *delay_max* with timeit, and call the decorated function:

```
decorated_func = timeit(delay_max)
print(decorated_func(3, 4, 1, delay=150))
```

The code should output 4, which is the return value of *delay_max(3, 4, 1, delay=150)*.
Since the function *delay_max* takes 150 ms to complete, the decorator should push a value close to 150 to the *execution_time* list.

Function Description

Complete the function *timeit* in the editor below. The function is a decorator with the above properties.

timeit has the following parameter:

func: a function

Constraints

- $1 \le n \le 20$
- $1 \le$ *args[i][j]* ≤ 100
- *function_name[i]* = {"delay_max", "delay_min", "delay_sum"}
- $50 \le$ *time_delay[i]* ≤ 400
- *time_delay[i]* is a multiple of 50.

▼ Input Format For Custom Testing

The first line contains an integer, *n*, denoting the number of function calls.

Each line *i* of the *n* subsequent lines (where $0 \le i < n$) contains space-separated values denoting *function_name[i], args[i],* and *time_delay[i]*.

▼ Sample Case 0

Sample Input For Custom Testing

```
3
delay_max 2 3 1 100
delay_sum 2 2 2 200
delay_min 4 5 5 300
```

Sample Output

```
3
6
4
[100, 200, 300]
```

Explanation

Here, there are 3 function calls

- *function_name* = "delay_max", *args* = [2, 3, 1], *time_delay* = 100, decorate the function *delay_max* with *timeit,* and call the decorated function with arguments 2, 3, and 1 with delay = 100. It should return 3. The decorator should push a value close to 100 to the list *execution_time.*

- *function_name* = "delay_sum", *args* = [2, 2, 2], *time_delay* = 200, decorate the function *delay_sum* with *timeit,* and call the decorated function with arguments 2, 2, and 2 with delay = 200. It should return 6. The decorator should push a value close to 200 to the list *execution_time.*
- *function_name* = "delay_min", *args* = [4, 5, 5], *time_delay* = 300, we decorate the function *delay_min* with *timeit,* and call the decorated function with arguments 4, 5, and 5 with delay = 300. It should return 4. The decorator should push a value close to 300 to the list *execution_time.*

Therefore, *execution_time* ≅ [100, 200, 300]

▼ Sample Case 1

Sample Input For Custom Testing

```
2
delay_max 2 8 1 150
delay_max 5 5 9 50
```

Sample Output

```
8
9
[150, 50]
```

Explanation

Here, there are 2 function calls

- *function_name* = "delay_max", *args* = [2, 8, 1], *time_delay* = 150, decorate the function *delay_max* with *timeit,* and call the decorated function with arguments 2, 8, and 1 with delay = 150. It should return 8. The decorator should push a value close to 150 to the list *execution_time.*
- *function_name* = "delay_max", *args* = [5, 5, 9], *time_delay* = 50, decorate the function *delay_max* with *timeit,* and call the decorated function with arguments 5, 5, and 9 with delay = 50. It should return 9. The decorator should push a value close to 50 to the list *execution_time.*

Therefore, *execution_time* ≅ [150, 50]

Question - 17

Function Composition

Create a function named *compose* that:

- Accepts any number of functions as arguments
- Returns a new function representing their composition
- Passes arguments to the first function
- Passes each function's output to the next function
- Returns the final output

The provided functions are:

- *add(*args)*: Returns the sum of all arguments
- *square(a)*: Returns the square of a number
- *splitter(a)*: Divides a number by 2 and returns [floor(*a*/2), *a*%2]
- *my_max(a)*: Returns the maximum value in a number or list
- *my_min(a)*: Returns the minimum value in a number or list

Note: Only the first function handles multiple arguments; subsequent functions take a single argument.

Example

functionsList = [add, splitter]

The function call *compose*(*functionList[1], functionList[2]*) should return a function, call it *composedFunctions.* If *argumentsList* = [2, 3], *composedFunctions*(2, 3), should return [2, 3].

- add(2, 3) -> 5
- splitter(5) -> [2, 3]

Function Description

Complete the function *compose* in the editor with the following parameters:

functionsList: function arguments

Returns

function: the composition of *functionsList*

Constraints

- $1 \le n \le 15$
- *functionsList[0] = add*
- *functionsList[i]* = {*square, splitter, my_max, my_min*}, where $i \ne 0$
- $1 \le q \le 10$, length of *argumentsList*
- $1 \le$ *argumentsList[i]* ≤ 10
- It is guaranteed that the list of functions generates a valid composition.

▼ Input Format For Custom Testing

The first line contains an integer, *n*, denoting the number of elements in *functionsList*.
Each line *i* of the *n* subsequent lines (where $0 \le i < n$) contains a string describing *functionsList[i]*.
The first line contains an integer, *q*, denoting the number of elements in *argumentsList*.
Each line *i* of the *q* subsequent lines (where $0 \le i < q$) contains an integer describing *argumentsList[i]*.

▼ Sample Case 0

Sample Input For Custom Testing

```
STDIN        Function
-----        --------
3            functionList[] size n = 3
add          functionList = ["add", "splitter", "my_max"]
splitter
my_max
3            argumentsList[] size q = 3
2            argumentsList = [2, 1, 3]
1
3
```

Sample Output

```
3
```

Explanation

compose(2, 1, 3) is processed as shown below -

- *add(2, 1, 3)* -> 6
- *splitter(6)* -> [3, 3]
- *my_max([3, 3])* -> 3

▼ Sample Case 1

Sample Input For Custom Testing

```
STDIN        Function
-----        --------
3            functionList[] size n = 3
add          functionList = ["add", "square", "splitter"]
square
splitter
2            argumentsList[] size q = 2
2            argumentsList = [2, 4]
4
```

Sample Output

```
[18, 18]
```

Explanation

compose(2, 4) is processed as shown below -

- *add(2, 4)* -> 6
- *square(6)* -> 36
- *splitter(36)* -> [18, 18]

Question - 18
Generator Context

Create a context manager class called *sequence_generator* that generates an infinite sequence of numbers. The sequence starts from a given value and increases by a specified step with each iteration.

The context manager should expose a *generate* method that yields numbers in the sequence.

Example

```
with sequence_generator(3, 5) as gen:
    for _ in range(3):
        print(next(gen), end=" ")
```

Output: 3 8 13

In this example, we create a sequence starting at 3 with a step of 5. We then retrieve the first three values from the sequence (3, 8, and 13) and print them.

Function Description
Complete the class *sequence_generator* in the editor below. It should be a context manager class and should expose *generate*.

sequence_generator's __init__() method has the following parameter(s):
int start: the start value
int step: the increment at each call

Constraints

- 1 ≤ *start, step* ≤ 100
- 1 ≤ *num* ≤ 10

▼ Input Format For Custom Testing

There are 3 lines of input, the lines contain *start, step,* and *num*.

▼ Sample Case 0

Sample Input For Custom Testing

```
STDIN    Function
-----    --------
22       start = 22
75       step = 75
6        num = 6
```

Sample Output

```
22
97
172
247
```

```
322
397
```

Explanation

Six numbers are generated, starting with 22 and adding 75 at each step.

▼ Sample Case 1

Sample Input For Custom Testing

```
STDIN    Function
-----    --------
45       start = 45
72       step = 72
1        num = 1
```

Sample Output

```
45
```

Explanation
The generator returns one number, starting with *45*. If more numbers were requested, *72* would be added at each step.

Question - 19
Custom Method Resolution

Implement a class *Child* that inherits from two already implemented classes, *Parent1* and *Parent2*. Both parent classes implement two methods: *fun1* and *fun2*.
In the *Child* class:

- *fun1* should call *Parent1*'s *fun1* before *Parent2*'s *fun1*
- *fun2* should call *Parent2*'s *fun2* before *Parent1*'s *fun2*

Example

```
class Parent1:
    def fun1(self):
        print("Parent1's fun1() method")

    def fun2(self):
        print("Parent1's fun2() method")


class Parent2:
    def fun1(self):
        print("Parent2's fun1() method")

    def fun2(self):
        print("Parent2's fun2() method")
```

Let *c* be the object of class *Child, c.fun1()* should print the following:

```
Parent1's fun1() method
Parent2's fun1() method
```

c.fun2() should print the following :

```
Parent2's fun2() method
Parent1's fun2() method
```

Function Description

Complete the class *Child* in the editor below with the methods as described.

Constraints

- $1 \le n \le 10$
- $1 \le arr[i] \le 2$

▼ Input Format For Custom Testing

The first line contains an integer, *n*, denoting the number of elements in *arr*.

Each line *i* of the *n* subsequent lines (where $0 \le i < n$) contains an integer describing *arr[i]*.

▼ Sample Case 0

Sample Input For Custom Testing

```
STDIN    Function
-----    --------
2        size of arr[] n = 2
1        arr = [1, 2]
2
```

Sample Output

```
Child fun1
Parent1 fun1
Parent2 fun1
Child fun2
Parent2 fun2
Parent1 fun2
```

Explanation

The first call is *c.fun1(),* and the second call is *c.fun2(),* where *c* is an object of class *Child.*

▼ Sample Case 1

Sample Input For Custom Testing

```
STDIN    Function
-----    --------
2        size of arr[] n = 2
2        arr = [2, 2]
2
```

Sample Output

```
Child fun2
Parent2 fun2
Parent1 fun2
Child fun2
Parent2 fun2
Parent1 fun2
```

Explanation

The first call is *c.fun2(),* and the second call is *c.fun2(),* where *c* is an object of class *Child.*

Question - 20

Python: Square Accumulate Root

A developer needs to perform three operations on input numbers: squaring a number, taking the square root of a number, or summing input numbers. The order of these operations is unknown beforehand, making it ideal for implementing co-routines using the producer-filter-consumer pattern.

Implement three co-routines:

- The accumulator: Receives a number, adds it to the previously stored sum (starting at 0), and yields the new sum.
- The squarer: Receives a number and yields its square.

- The rooter: Receives a number and yields the floor of its square root.

Example

order = `[square, accumulate]`

nums = [1, 2, 3]

- After processing 1: The output is 1 (1^2 = 1, accumulate to 0 + 1 = 1)
- After processing 2: The output is 5 (2^2 = 4, accumulate to 1 + 4 = 5)
- After processing 3: The output is 14 (3^2 = 9, accumulate to 5 + 9 = 14)

Functions Description

Complete the co-routine *accumulator*, *squarer*, and *rooter* in the editor below.

These co-routines do not have any input and communicate completely through the sub-routine pipeline.

Constraints

- $1 \le n \le 10^5$
- $1 \le nums[i] \le 10^5$ (where $0 \le i < n$)

▼ Input Format For Custom Testing

The first line contains a string, *order*, describing the order in which to perform the operations.

The next line contains an integer, *n*, denoting the number of elements in *nums*.

Each line *i* of the *n* subsequent lines (where $0 \le i < n$) contains an integer describing *nums[i]*.

▼ Sample Case 0

Sample Input For Custom Testing

```
STDIN                     Function
-----                     --------
[square, accumulate]    → string order = '[square, accumulate]'
3                       → integer n = 3
1                       → nums[] = [ 1, 2, 3 ]
2
3
```

Sample Output

```
1
5
14
```

Explanation

The order of the function shows that we need to square a number and then accumulate the result.

- square 1 = 1, accumulate = 0 + 1 = 1
- square 2 = 4, accumulate = 1 + 4 = 5
- square 3 = 9, accumulate = 5 + 9 = 14

▼ Sample Case 1

Sample Input For Custom Testing

```
STDIN                         Function
-----                         --------
[root, square, accumulate]  → string order = '[root, square, accumulate]'
5                           → integer n = 5
3                           → nums[] = [ 3, 6, 1, 2, 3 ]
6
1
2
3
```

Sample Output

```
1
5
6
7
8
```

Explanation

We need to take the square root of the input number first, then square the result, then accumulate.

- root 3 = 1, square 1 = 1, accumulate = 0 + 1 = 1
- root 6 = 2, square 2 = 4, accumulate = 1 + 4 = 5
- root 1 = 1, square 1 = 1, accumulate = 5 + 1 = 6
- root 2 = 1, square 1 = 1, accumulate = 6 + 1 = 7
- root 3 = 1, square 1 = 1, accumulate = 7 + 1 = 8

Question - 21

Python: Non-Primes Generator

Given an integer *k*, print the first *k* non-prime positive integers, each on a new line.

Example

For *k* = 10, the output would be:

```
1
4
6
8
9
10
12
14
15
16
```

Function Description

Complete the function *manipulate_generator* in the editor with the following parameter(s):

g: a generator

n: an integer

The function must manipulate the generator function so that the first *k* non-prime positive integers are printed, each on a separate line.

Constraints

- $1 \le k \le 10^5$

▼ Input Format For Custom Testing

The only line contains a single integer *k,* the number of non-primes to print.

▼ Sample Case 0

Sample Input 0

```
STDIN        Function
-----        --------
12        →  integer k = 12
```

Sample Output 0

```
1
4
6
```

```
8
9
10
12
14
15
16
18
20
```

Explanation 0

The output contains the first 12 non-prime positive integers.